MESTRADO PROFISSIONAL EM COMPUTAÇÃO APLICADA

Estruturas de Dados e Análise de Algoritmos

P2 Resolvida

Estruturas de Dados na Linha de Produção (BigO Motors)

Material de estudo/preparação para provas futuras, a partir de uma prova (P2) já aplicada em turma anterior, com apoio de IA (Claude). Cada questão usa um CPF ilustrativo (123.456.789-09) como dado de exemplo, claramente identificado como tal — não é um documento real. Todo código Java abaixo foi realmente compilado e executado nesta sessão (Java 21); as saídas mostradas são reais, não fabricadas.

Cenário da fábrica, em resumo

A BigO Motors organiza a produção de veículos com: ordem de entrada planejada; retrabalho que retorna ao ponto exato do defeito; peças com busca rápida por código; veículo decomposto em sistemas/subsistemas; checkpoints de aprovação binária; setores conectados por rotas; distribuição de peças por proximidade; e investigação retroativa de falhas.

Mapeamento das estruturas ao cenário

Estrutura/algoritmo	Onde se aplica	Motivo em uma frase
Pilha	Retrabalho de um veículo	Volta ao último ponto interrompido (LIFO)
Fila	Entrada de veículos	Ordem de chegada preservada (FIFO)
Lista ligada	Componentes do veículo	Tamanho variável, insere/remove fácil
Árvore	Sistemas/subsistemas	Relação pai-filho
Árvore binária	Checkpoint pass/fail	No máximo 2 saídas por decisão
BST	Busca de peça por código	Busca ordenada e dinâmica
Árvore AVL	Inventário em grande escala	Mantém busca rápida mesmo crescendo muito
Grafo	Setores interligados	Conexões muitos-para-muitos
BFS	Distribuição por proximidade	Explora por camadas de distância
DFS	Investigação de falha	Segue um caminho até a origem

Nas dez questões a seguir, os dígitos do CPF ilustrativo (1,2,3,4,5,6,7,8,9,0,9) viram apenas identificadores numéricos para dar exemplos concretos.

1. Fila — entrada de veículos

Estrutura: Fila (Queue).

Por que: A ordem de chegada deve ser mantida — primeiro que entra é o primeiro a ser montado (FIFO). Uma pilha inverteria a ordem.

Custo das operações: Entrar (enqueue) e sair (dequeue) são O(1) com ArrayDeque.

```
Queue<Integer> filaVeiculos = new ArrayDeque<>();
int[] cpf = {1, 2, 3, 4, 5, 6};
for (int id : cpf) filaVeiculos.offer(id);
while (!filaVeiculos.isEmpty()) {
    int proximo = filaVeiculos.poll();
    System.out.println("monta veiculo " + proximo);
}
```

```
enqueue(veiculo 1) -> fila: [1]
enqueue(veiculo 2) -> fila: [1, 2]
enqueue(veiculo 3) -> fila: [1, 2, 3]
enqueue(veiculo 4) -> fila: [1, 2, 3, 4]
enqueue(veiculo 5) -> fila: [1, 2, 3, 4, 5]
enqueue(veiculo 6) -> fila: [1, 2, 3, 4, 5, 6]

dequeue() -> monta veiculo 1 | fila restante: [2, 3, 4, 5, 6]
dequeue() -> monta veiculo 2 | fila restante: [3, 4, 5, 6]
dequeue() -> monta veiculo 3 | fila restante: [4, 5, 6]
dequeue() -> monta veiculo 4 | fila restante: [5, 6]
dequeue() -> monta veiculo 5 | fila restante: [6]
dequeue() -> monta veiculo 6 | fila restante: []
```

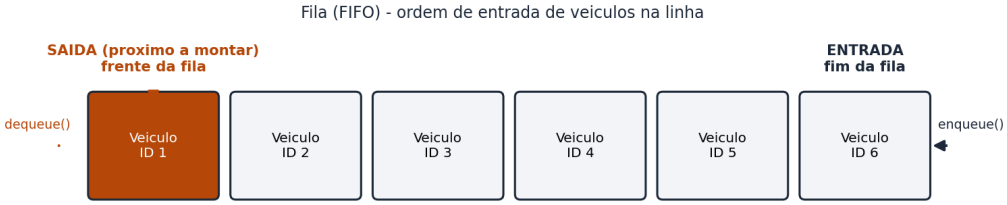


Figura 1 — fila FIFO: veículo 1 é o próximo a ser retirado.

2. Pilha — gerenciamento de retrabalho

Estrutura: Pilha (Stack).

Por que: O retrabalho deve retomar exatamente do último defeito tratado — o último a entrar é o primeiro a sair (LIFO), como o botão 'desfazer' de um editor.

Custo das operações: push e pop são O(1) com ArrayDeque usado como pilha.

```
Deque<String> pilhaRetrabalho = new ArrayDeque<>();
int[] etapas = {7, 8, 9};
for (int etapa : etapas) pilhaRetrabalho.push("etapa " + etapa);
while (!pilhaRetrabalho.isEmpty()) {
    System.out.println("retoma " + pilhaRetrabalho.pop());
}
```

```
push(etapa 7) -> pilha: [etapa 7]
push(etapa 8) -> pilha: [etapa 8, etapa 7]
push(etapa 9) -> pilha: [etapa 9, etapa 8, etapa 7]

pop() -> retoma etapa 9 | pilha restante: [etapa 8, etapa 7]
pop() -> retoma etapa 8 | pilha restante: [etapa 7]
pop() -> retoma etapa 7 | pilha restante: []
```

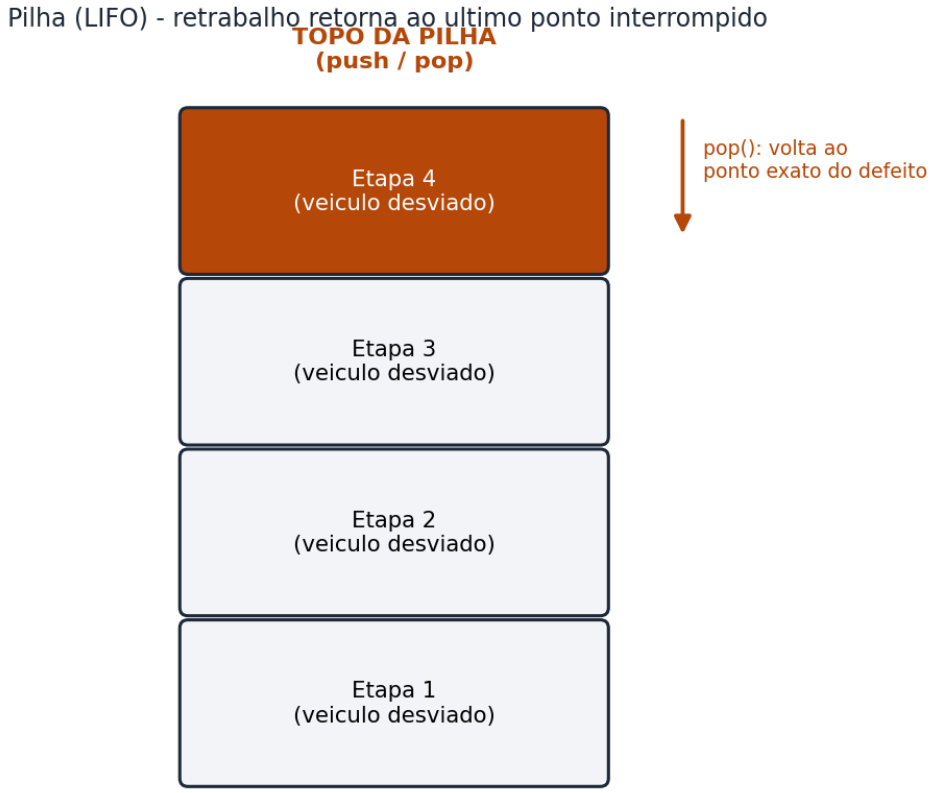


Figura 2 — pilha LIFO: retrabalho retoma sempre o topo.

3. Lista ligada — componentes por veículo

Estrutura: Lista ligada (LinkedList).

Por que: O conjunto de componentes muda de tamanho com frequência (opcionais adicionados/removidos) — lista ligada evita realocar um array inteiro a cada mudança.

Custo das operações: Inserir/remover numa posição já localizada: O(1). Buscar um componente específico: O(n) (precisa percorrer).

```
LinkedList<String> componentes = new LinkedList<>();
componentes.addLast("Motor(cod1)");
componentes.addLast("Cambio(cod2)");
componentes.addLast("Painel(cod3)");
componentes.addLast("Bancos(cod4)");
componentes.addLast("Rodas(cod5)");
componentes.addLast("OpcionalTetoSolar(cod6)");
componentes.remove("Painel(cod3)");
componentes.add(1, "FiltroAr(cod7)");
```

```
Lista inicial: [Motor(cod1), Cambio(cod2), Painel(cod3), Bancos(cod4), Rodas(cod5)]
Apos addLast (O(1)): [Motor(cod1), Cambio(cod2), Painel(cod3), Bancos(cod4), Rodas(cod5), OpcionalTetoSolar(coc
Apos remover Painel por valor (O(n) para localizar): [Motor(cod1), Cambio(cod2), Bancos(cod4), Rodas(cod5), Opc
Apos inserir na posicao 1 (O(1) apos localizar o no): [Motor(cod1), FiltroAr(cod7), Cambio(cod2), Bancos(cod4),
```

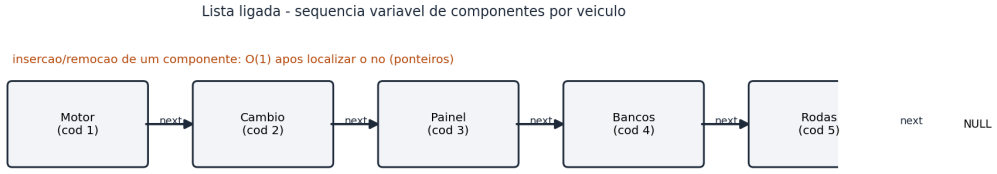


Figura 3 — lista ligada de componentes com inserção O(1) ao final.

4. Árvore — estrutura de montagem

Estrutura: Árvore (Tree).

Por que: Veículo → sistemas → peças é uma decomposição hierárquica clássica (relação pai-filho), que uma árvore representa diretamente.

Custo das operações: Percorrer a árvore inteira: O(n) nós. Localizar um subsistema a partir da raiz: O(altura da árvore).

```
No veiculo = new No("Veiculo");
No motor = new No("Motor").add(new No("Pistao")).add(new No("Virabrequim"));
No carroceria = new No("Carroceria").add(new No("Chassi")).add(new No("Portas"));
No eletronica = new No("Eletronica").add(new No("ECU")).add(new No("Chicote"));
veiculo.add(motor).add(carroceria).add(eletronica);
```

```
Veiculo
  Motor
    Pistao
    Virabrequim
  Carroceria
    Chassi
    Portas
  Eletronica
    ECU
    Chicote
Total de nos na arvore: 10
```

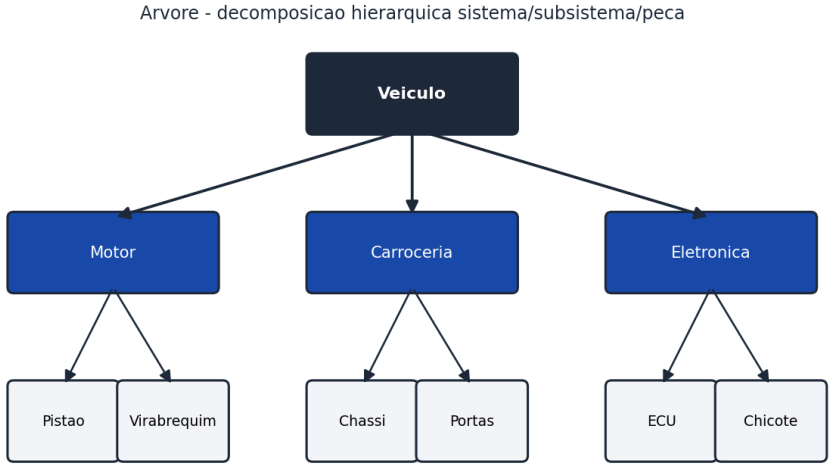


Figura 4 — árvore de decomposição sistema/subsistema/peça.

5. Árvore binária — decisão pass/fail

Estrutura: Árvore binária.

Por que: Cada checkpoint tem no máximo 2 saídas (aprovado/reprovado), e uma reprovação pode se desdobrar em outra decisão binária (recuperável/irrecuperável).

Custo das operações: Percorrer do topo até uma folha: O(altura da árvore de decisão) — normalmente pequena e fixa em um processo real.

```
Decisao geral = new Decisao("InspecaoGeral");
geral.aprovado = new Decisao("PASS");
geral.reprovado = new Decisao("FAIL");
geral.reprovado.aprovado = new Decisao("Retrabalho(cod9)");
geral.reprovado.reprovado = new Decisao("Descarte(cod0)");
```

```
Veiculo cod 9 (reprova, depois recuperavel): InspecaoGeral -> FAIL -> Retrabalho(cod9)
Veiculo cod 0 (reprova, depois irrecuperavel): InspecaoGeral -> FAIL -> Descarte(cod0)
Veiculo aprovado direto: InspecaoGeral -> PASS
```

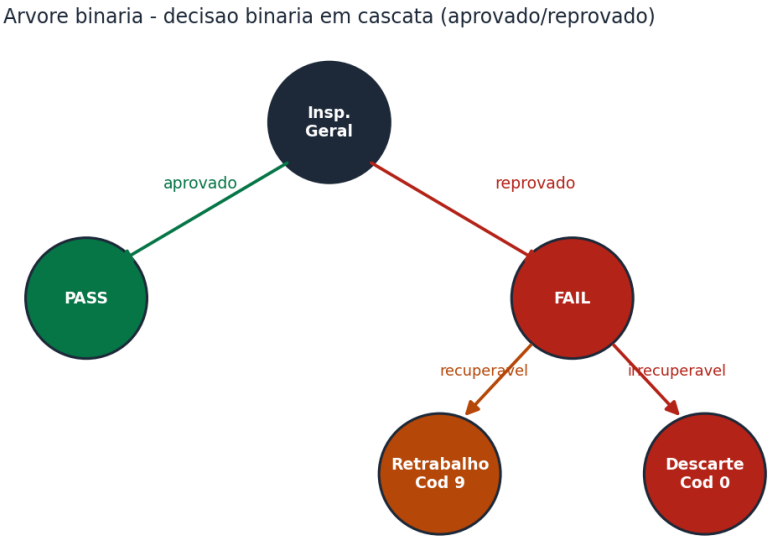


Figura 5 — árvore binária de decisão pass/fail.

6. Árvore binária de busca — peças por código

Estrutura: Árvore binária de busca (BST).

Por que: Cada peça tem um código único; a BST mantém os códigos ordenados e permite busca eficiente mesmo com o inventário mudando (entra/sai peça).

Custo das operações: Busca, inserção e remoção: O(log n) se a árvore estiver balanceada; O(n) se degenerar (ver questão 7).

```
int[] codigos = {5, 2, 8, 1, 4, 7, 9};
No raiz = null;
for (int c : codigos) raiz = inserir(raiz, c);
buscar(raiz, 7);  // conta comparacoes
```

```
busca(7) -> encontrado=true,  comparacoes=3
busca(1) -> encontrado=true,  comparacoes=3
busca(3) -> encontrado=false, comparacoes=3
```

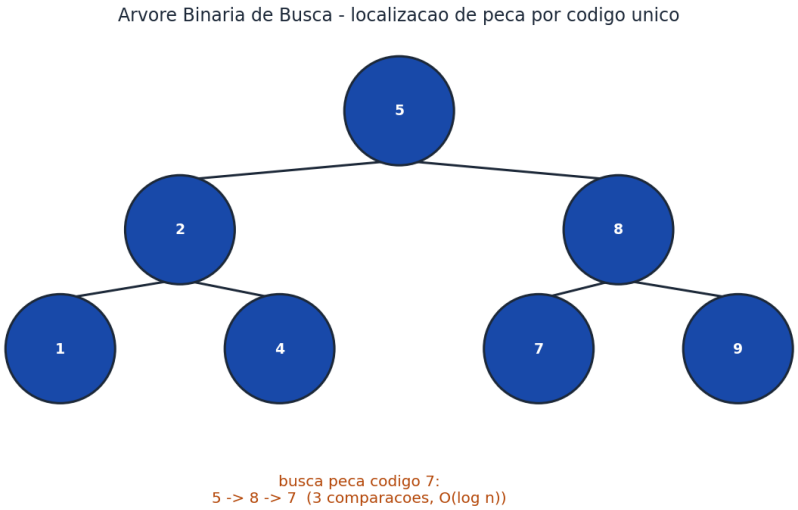


Figura 6 — BST: busca da peça de código 7 em 3 comparações.

7. Árvore AVL — eficiência em grande volume

Estrutura: Árvore AVL.

Por que: Inserir códigos em ordem crescente numa BST comum degenera a árvore em uma lista (busca vira O(n)). A AVL rebalanceia automaticamente após cada inserção/remoção (rotações), mantendo a altura sempre próxima de log2(n).

Custo das operações: Busca, inserção e remoção: O(log n) garantido, mesmo no pior padrão de entrada.

```
int[] codigos = {1, 2, 3, 4, 5, 6, 7, 9};  // pior caso para BST comum
No raizBST = null;
for (int c : codigos) raizBST = inserirBST(raizBST, c);

No raizAVL = null;
for (int c : codigos) raizAVL = inserirAVL(raizAVL, c);  // com rotacoes
```

```
BST comum, insercoes em ordem crescente [1, 2, 3, 4, 5, 6, 7, 9]
Altura da BST comum: 8 (degenerada, busca O(n))
Altura da AVL apos as mesmas insercoes: 4 (mantida O(log n))
```

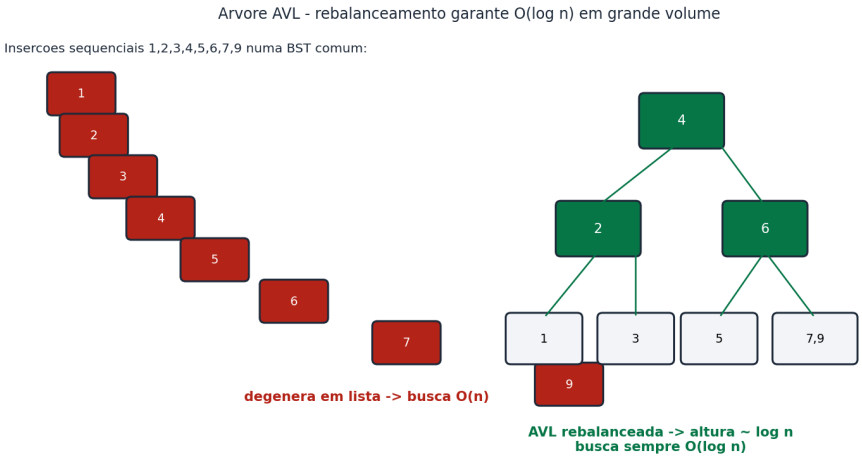


Figura 7 — BST degenerada (altura 8) vs. AVL rebalanceada (altura 4).

8. Grafo — fábrica como sistema interligado

Estrutura: Grafo.

Por que: Os setores se conectam de forma arbitrária (não hierárquica) e podem formar ciclos (ex.: inspeção retorna a recebimento) — só um grafo modela isso corretamente.

Custo das operações: Percorrer o grafo inteiro (BFS/DFS): O(V+E), onde V é o número de setores e E o número de rotas.

```
addRota(setores, "Recebimento", "Estamparia");
addRota(setores, "Estamparia", "Solda");
addRota(setores, "Solda", "Pintura");
addRota(setores, "Pintura", "Montagem");
addRota(setores, "Montagem", "Inspecao");
addRota(setores, "Inspecao", "Recebimento");  // ciclo
addRota(setores, "Montagem", "Expedicao");
addRota(setores, "Pintura", "Expedicao");
```

```
V (setores) = 7
E (rotas) = 8
Recebimento -> [Estamparia, Inspecao]
Estamparia  -> [Recebimento, Solda]
Solda       -> [Estamparia, Pintura]
Pintura     -> [Solda, Montagem, Expedicao]
Montagem    -> [Pintura, Inspecao, Expedicao]
Inspecao    -> [Montagem, Recebimento]
Expedicao   -> [Montagem, Pintura]
```

Grafo - setores da fabrica interligados por rotas de fluxo

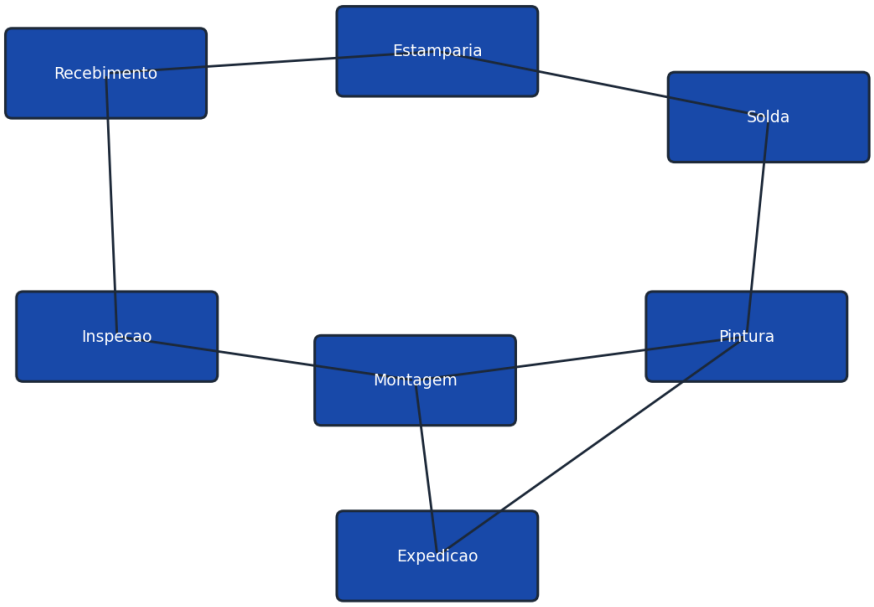


Figura 8 — grafo de setores da fábrica interligados por rotas.

9. BFS — distribuição de peças por proximidade

Algoritmo: Busca em largura (BFS).

Por que: BFS visita primeiro tudo que está a 1 rota de distância do depósito, depois a 2 rotas, e assim por diante — exatamente a ordem certa para entregar por proximidade.

Custo: O(V+E), mesma ordem de percorrer o grafo inteiro.

```
Queue<String> fila = new ArrayDeque<>();
fila.add("DepositoCentral");
distancia.put("DepositoCentral", 0);
while (!fila.isEmpty()) {
    String atual = fila.poll();
    for (String vizinho : rede.get(atual)) {
        if (!distancia.containsKey(vizinho)) {
            distancia.put(vizinho, distancia.get(atual) + 1);
            fila.add(vizinho);
        }
    }
}
```

```
Ordem de entrega (BFS a partir do deposito central):
  nivel 0: DepositoCentral
  nivel 1: Solda
  nivel 1: Pintura
  nivel 1: Montagem
  nivel 2: Estacao1
  nivel 2: Estacao2
  nivel 2: Estacao3
  nivel 2: Estacao4
```

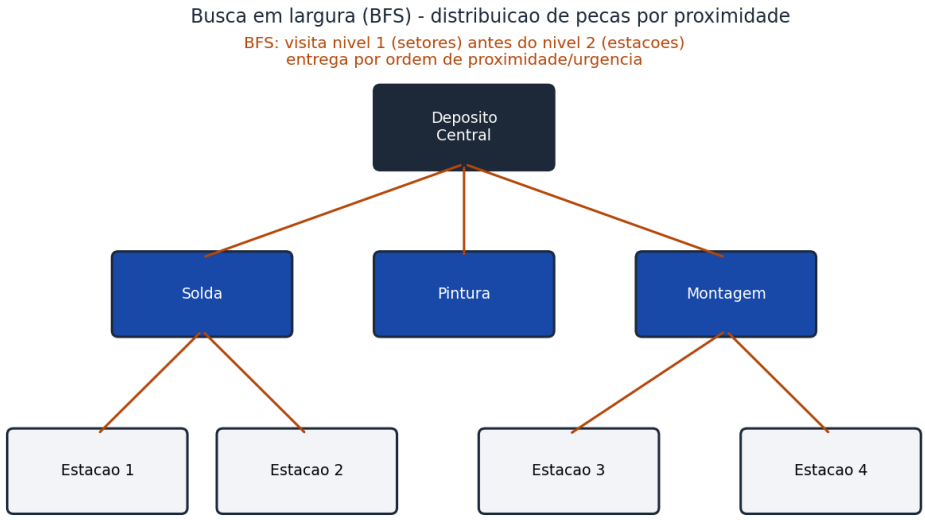


Figura 9 — BFS a partir do depósito central, por nível de proximidade.

10. DFS — investigação retroativa de falha

Algoritmo: Busca em profundidade (DFS).

Por que: A investigação segue um único caminho (o histórico do veículo) de trás para frente até achar a origem — não precisa explorar por camadas, precisa ir fundo em um caminho.

Custo: O(V+E) no pior caso (percorre todo o histórico até a origem).

```
static void dfs(String atual, Map<String, List<String>> g, Set<String> visitados) {
    if (visitados.contains(atual)) return;
    visitados.add(atual);
    System.out.println("investigando: " + atual);
    for (String proximo : g.get(atual)) dfs(proximo, g, visitados);
}
```

```
DFS retroativo a partir da falha em Expedicao:
  investigando: Expedicao
  investigando: InspecaoFinal
  investigando: Montagem
  investigando: Pintura
  investigando: Solda
Causa raiz identificada: Solda
```



Figura 10 — DFS retrocedendo etapa a etapa até a causa raiz.